Computergrafik und Computeranimation

Arnd Vitzthum

2. 2D-Vektorgrafik

…

2.4 Zeichnen von Linien

2.5 Zeichnen von Kreisen und Kurven

2.6 Linienstile, dicke Linien und Antialiasing

…

Basiert teilweise auf Vorlesungsmaterial von Dr. Axel Hoppe, LMU München

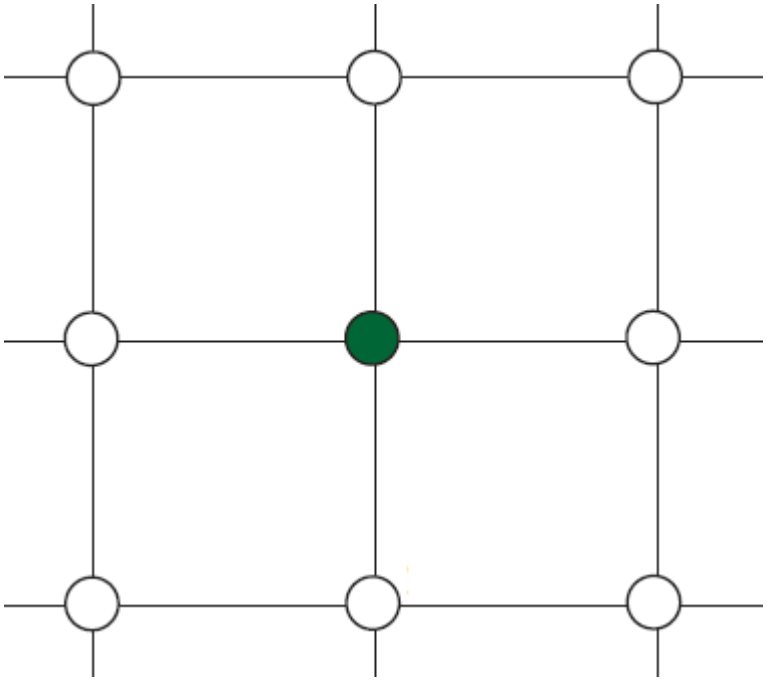
Abstraktes Pixelmodell für Algorithmenentwurf

Ein Pixel …

… ist ein Kreuzungspunkt auf einem gleichmäßigen Gitter.

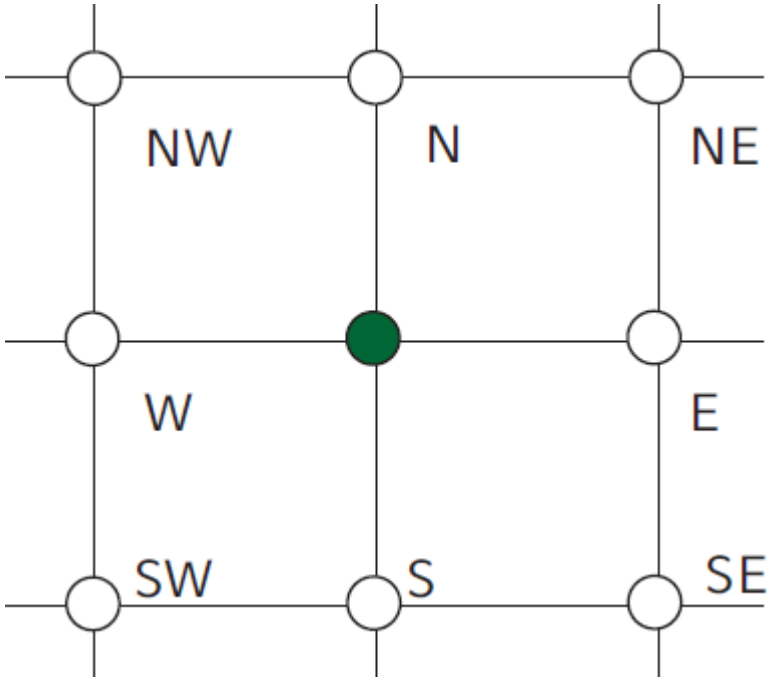
… hat keine Ausdehnung.

… ist an oder aus.

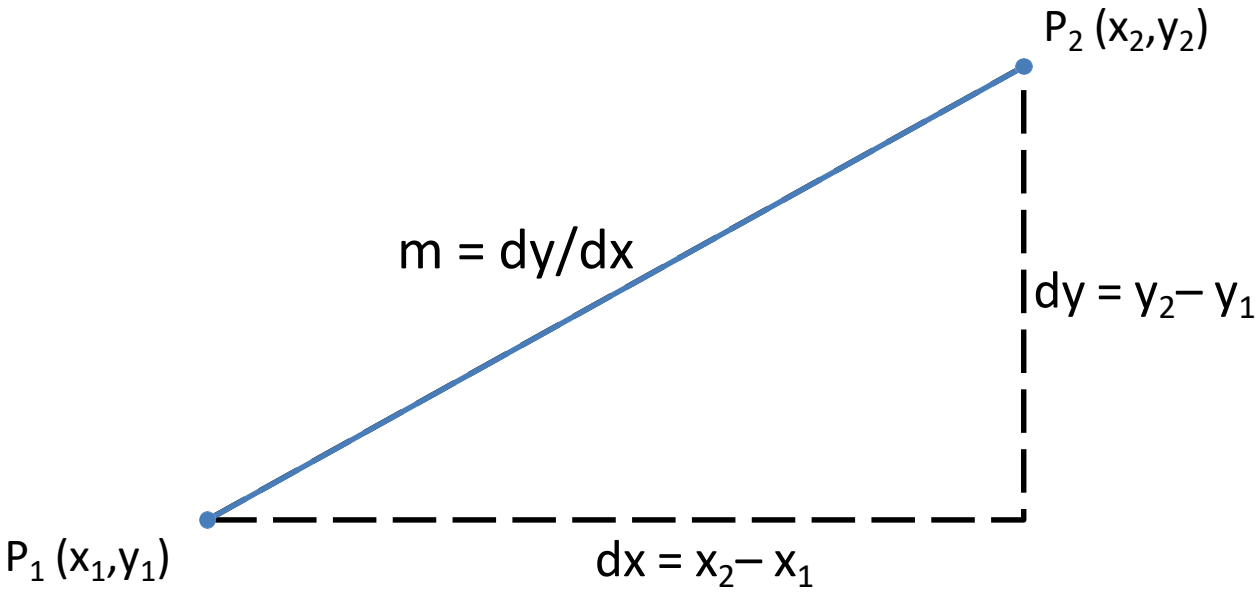


Pixelnachbarschaften

- Vierernachbarschaft
- Achternachbarschaft



Linienanstieg m



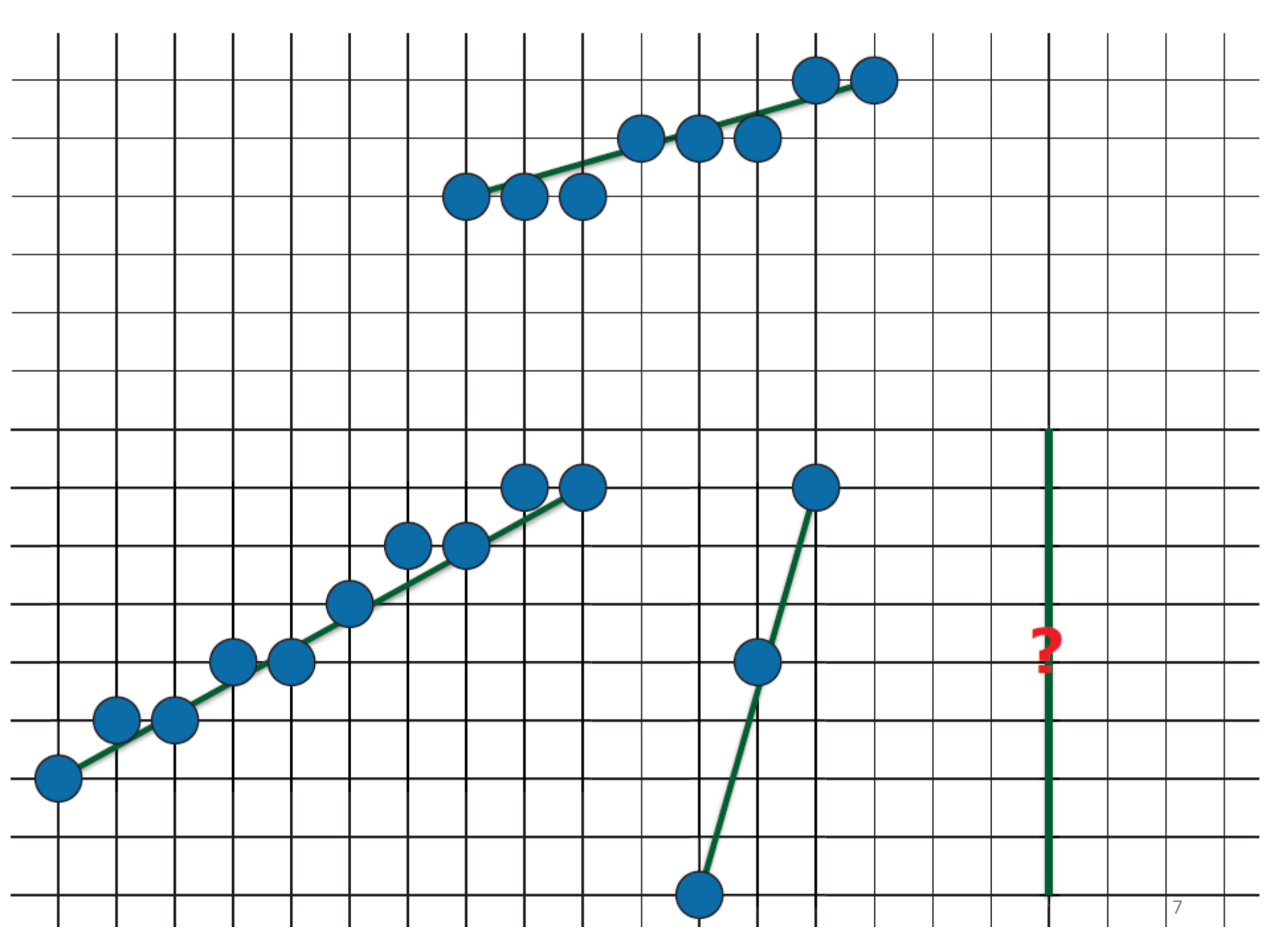
Naiver Algorithmus

- Gegeben: $P_1=(x_1, y_1)$ und $P_2=(x_2, y_2)$; jeweils Integer-Koordinaten
- Gesucht: Alle Pixel auf der Linie
- Lösungsansatz:

```
double m = (y2-y1)/(x2-x1) ;
for (int x = x1; x <= x2; x++) {
    double y = m*x + b;
    setPixel( x, round( y ));
}
```

Naiver Algorithmus – Effizienz

```
double m = (y2-y1)/(x2-x1);
for (int x = x1; x <= x2; x++) {
  double y = m*x + b;
  setPixel( x, round( y ));
}
```



Bresenham-Algorithmus (auch: *Midpoint-Line-Alg., Mittelpunktschema)*

Vorteile:

☺ Ausschließlich Integer-Arithmetik

☺ Keine Division

☺ Keine komplizierten Multiplikationen

Einschränkungen:

☹ $dy \leq dx$ → Anstieg $m = dy/dx$ zwischen einschließlich 0 und 1

➢ Erweiterung auf andere Anstiege aber (leicht) möglich!

☹ Linie muss von links nach rechts gezeichnet werden

➢ Ggf. Start- und Endpunkt vertauschen

Vorüberlegungen

- Explizite Darstellung der Geradengleichung:

  $y = mx+b$

  $y= (dy/dx)*x+b$

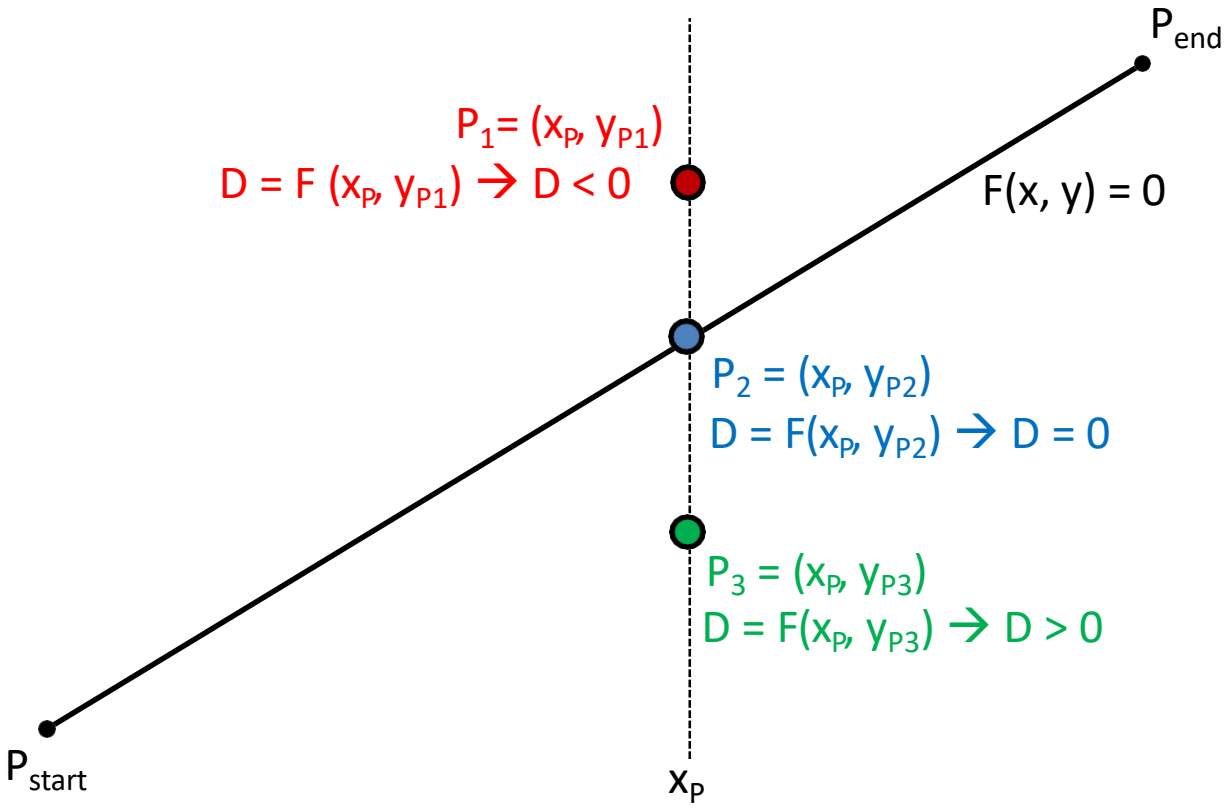
➢ Implizite Darstellung der Geradengleichung:

$dy*x – dx*y + dx*b = 0$

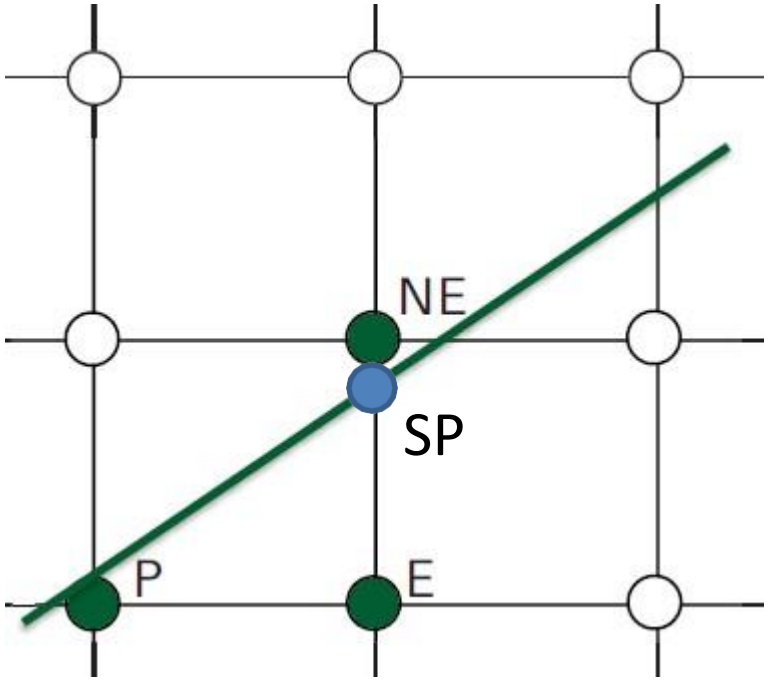
$Ax + By + C = 0$ mit $A = dy$ und $B = -dx$

A und B sind ganzzahlig!

„Entscheidungsvariable“
D = F(x, y) = dy*x – dx*y + dx*b

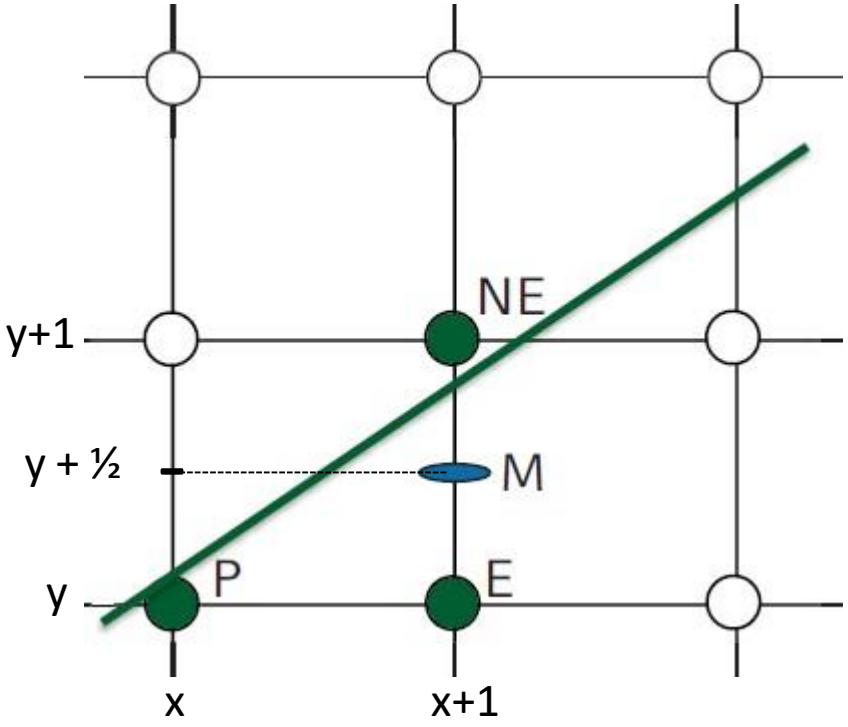


Bestimmung des nächsten zu zeichnenden Pixels: Erster Ansatz



- Aktuelles Pixel: P
- Nächstes Pixel E od. NE, da $0 \leq m \leq 1$
- ➢ Entscheide, ob E od. NE näher bei SP
- Das für jedes Pixel

Bestimmung des nächsten zu zeichnenden Pixels: Verbesserung durch ***Mittelpunktschema***



$$M = \left(x + 1, y + \frac{1}{2}\right)$$

$$D = F\left(x + 1, y + \frac{1}{2}\right)$$

D < 0 → E (M „über“ Linie)

D > 0 → NE (M „unter“ Linie)

D = 0 → E o. NE (M auf Linie)

Mittelpunktschema, erster Algorithmus-Entwurf

1. Zeichne Startpixel
2. Berechne D für nächstes zu zeichnendes Pixel
   (nächsten Mittelpunkt in Entscheidungsfunktion einsetzen)
3. Wenn D > 0 zeichne NE, sonst zeichne E
4. Gehe zu 2

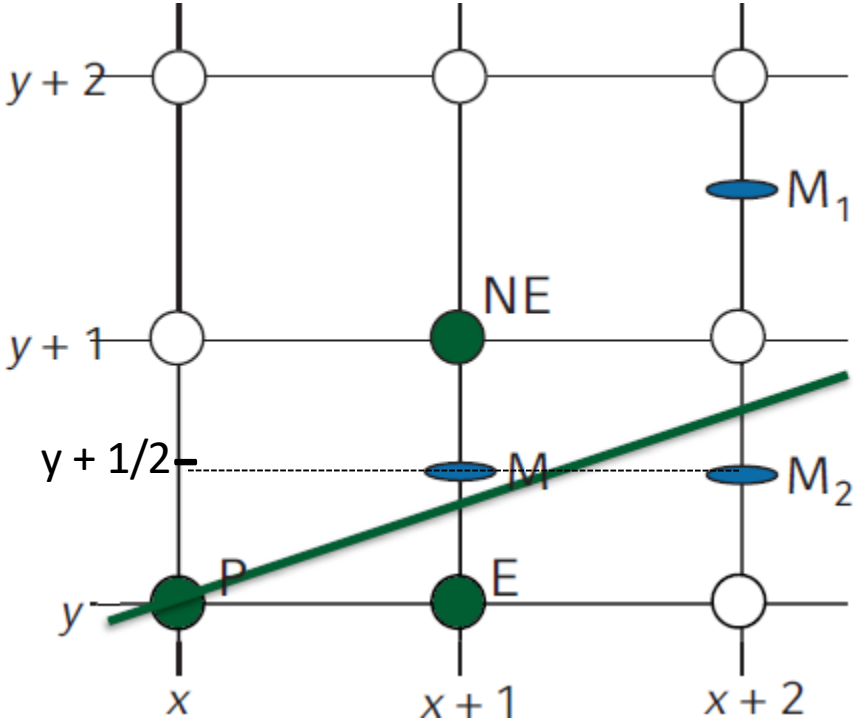
- Vereinfachte Berechnung von D durch **inkrementelles Vorgehen**?

$$D_{neu} = D_{alt} + \boldsymbol{\Delta}$$

- Berechnung der Inkremente **Δ** :

$$\boldsymbol{\Delta} = D_{neu} - D_{alt}$$

Bestimmung von **Δ** (Fall 1): E ausgewählt, M_2 nächster Mittelpunkt



$\Delta_E = D_{neu} - D_{alt}$

$\Delta_E = F(x_{M2}, y_{M2}) - F(x_M, y_M)$

$\Delta_E = F(x+2, y+1/2) - F(x+1,y+1/2)$

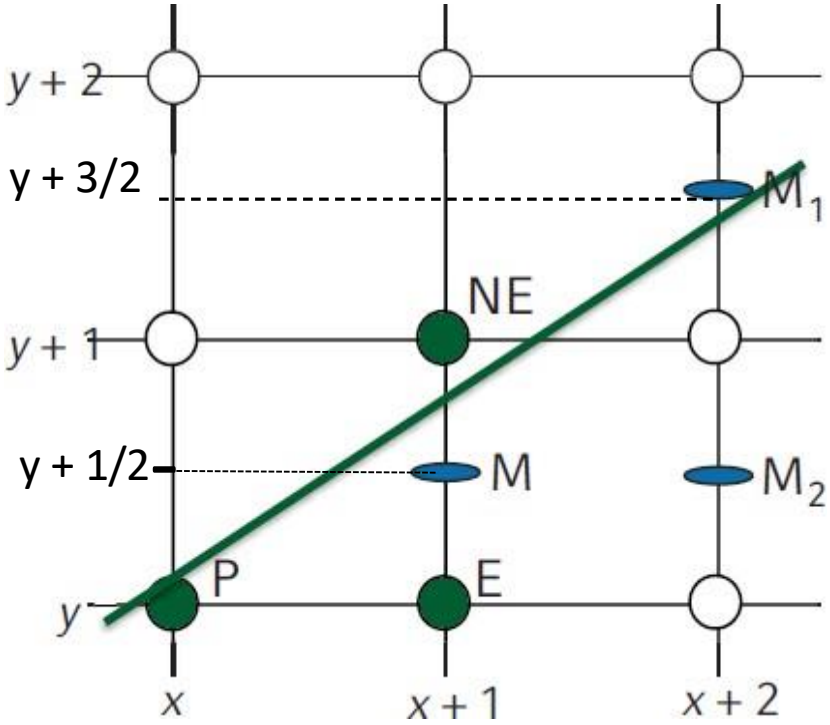
$D_{neu} = A\cdot(x+2) + B\cdot(y+1/2) + C$

$D_{alt} = A\cdot(x+1) + B\cdot(y+1/2) + C$

$\Delta_E = A$

$\mathbf{\Delta_E = dy}$; $\Delta_E \geq 0$

Bestimmung von **Δ** (Fall 2):
NE ausgewählt, M_1 nächster Mittelpunkt



$\Delta_{NE} = D_{neu} - D_{alt}$

$\Delta_{NE} = F(x_{M1}, y_{M1}) - F(x_M, y_M)$

$\Delta_{NE} = D_{neu} - D_{alt}$

$\Delta_{NE} = F(x+2, y+3/2) - F(x+1, y+1/2)$

$D_{neu} = A\cdot(x+2) + B\cdot(y+3/2) + C$

$D_{alt} = A\cdot(x+1) + B\cdot(y+1/2) + C$

$\Delta_{NE} = A + B$

$\mathbf{\Delta_{NE} = dy - dx}$

$dy \leq dx \rightarrow \Delta_{NE} \leq 0$

Initialwert der *Entscheidungsvariablen* D

- Startpunkt $P_0 = (x_0, y_0)$ befindet sich auf der Linie: $Ax_0 + By_0 + C = 0$
- Erster Mittelpunkt: $M = (x_0 + 1, y_0 + ½)$

$D_{init} = F(x_0 + 1, y_0 + ½)$

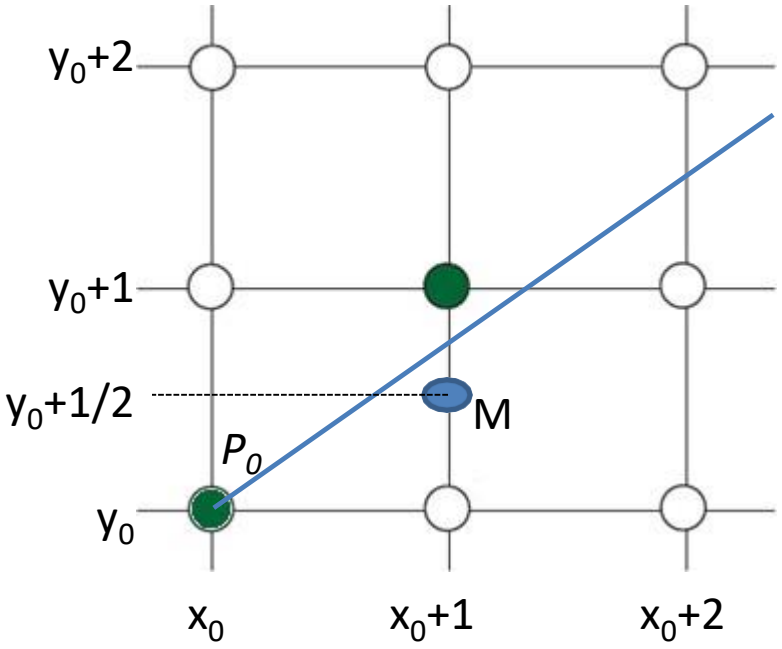
$D_{init} = A \cdot (x_0 + 1) + B \cdot (y_0 + ½) + C$

$D_{init} = Ax_0 + A + By_0 + ½B + C$

$D_{init} = Ax_0 + By_0 + C + A + ½B$

$D_{init} = A + ½B$

$\mathbf{D_{init} = dy - ½dx}$



Mittelpunktschema – Algorithmus (1)

1. Berechne dx und dy mittels Start- und Endpunkt
2. Berechne den Initialwert für D sowie Δ_E und Δ_{NE}
3. Setze x und y auf die Koordinaten des Startpunktes
4. Zeichne Startpixel
5. Entscheide mittels D, ob E oder NE das nächste Pixel ist
6. Aktualisiere x und y entsprechend der Wahl von E oder NE und zeichne das Pixel an der Position (x, y)
7. Aktualisiere D entsprechend der Wahl von E oder NE
8. Weiter mit 5., solange der Endpunkt nicht erreicht ist

☹ *D ist nicht zwingend eine Ganzzahl, da Initialwert* ***D = dy – ½dx***

➢ *Vereinfachte Berechnung durch Ganzzahlen möglich?*

Ganzzahlige *Entscheidungsvariablen*

- Beobachtung: Entscheidend ist **nur** das Vorzeichen von D

➢ Neue Entscheidungsvariable: $d = 2 \cdot \mathrm{D} = 2 \cdot F\left(x_0 + 1, y_0 + \frac{1}{2}\right)$

➢ Vorzeichen von d und D stimmen überein!

➢ Initialwert ganzzahlig: $d = 2 \cdot \mathrm{D} = 2 \cdot F\left(x_0 + 1, y_0 + \frac{1}{2}\right) = 2dy - dx$ ☺

Anpassung der Inkremente

Bekannt:

$\Delta = D_{neu} - D_{alt}$

$d = 2 \cdot D$

Ansatz:

$\Delta' = d_{neu} - d_{alt}$

$\Delta' = 2 \cdot D_{neu} - 2 \cdot D_{alt}$

$\Delta' = 2 \cdot (D_{neu} - D_{alt})$

$\Delta' = 2 \cdot \Delta$

Angepasste Inkremente:

$\boldsymbol{\Delta'_{NE} = 2 \cdot (dy - dx)}$

$\boldsymbol{\Delta'_{E} = 2 \cdot dy}$

Mittelpunktschema – Algorithmus (2)

```
void midpoint-line(int xo, int yo, int xe, int ye, int color)
{
    int dx = xe - xo;
    int dy = ye - yo;
    int d = 2 * dy - dx;
    int incrE = 2 * dy;
    int icrNE = 2 * (dy - dx);
    int x = xo;
    int y = yo;
    setPixel(xo, yo, color);
    while (x<xe) {
        if (d<=0) {
            d+=incrE;
            x++;
        } else {
            d += incrNE;
            x++; y++;
        }
        setPixel(x, y, color);
    }
}
```

```
void midpoint-line(int xo, int yo,
  int xe, int ye, int color)
{
    int dx = xe - xo;
    int dy = ye - yo;
    int d = 2 * dy - dx;
    int incrE = 2 * dy;
    int icrNE = 2 * (dy - dx);
    int x = xo;
    int y = yo;
    setPixel(xo, yo, color);
    while (x<xe) {
          if (d<=0) {
                d+=incrE;
                x++;
          } else {
                d += incrNE;
                x++; y++;
          }
          setPixel(x, y, color);
    }
}
```

Beispiel: Linie von (0,0) nach (5,3)

xo=0, yo=0, xe=5, ye=3

dx=5-0=5

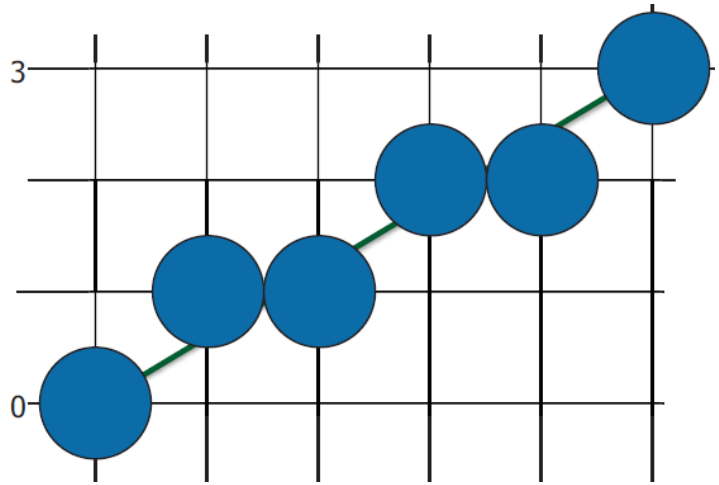
dy=3-0=3

d=2*3-5=1

incE=2*3=6

incNE=2*(3-5)=-4

d (alt)	d (alt) ≤ 0?	d (neu)	x	y
1	n	1-4=-3	1	1
-3	j	-3+6=3	2	1
3	n	3-4=-1	3	2
-1	j	-1+6=5	4	2
5	n	5-4=1	5	3



Vorteile des Mittelpunkt-Algorithmus‘

☺ Integer-Arithmetik

☺ Einfache inkrementelle Pixelberechnung (Additionen)

☺ Verdopplung eines Wertes per Bitverschiebung realisierbar

☺ Kleine Schleifenkörper

☺ Implementierung in Hardware einfach

Nachteile/Erweiterungen

- ☹ Beschränkung des Anstiegs
  - ☺ Algorithmus leicht auf beliebige Anstiege erweiterbar (Vertauschen von x und y, Vorzeichen, etc.)

- ☹ Lange Linien werden Pixel für Pixel gezeichnet
  - ➢ Erweiterungen, die gleich zwei Pixel im Voraus berechnen
  - ➢ Linie setzt sich aus „Mustern“ zusammen → Muster finden und als Gesamtheit setzen